

Formalization of Gödel’s Incompleteness Theorems in Basic Recursive Arithmetic (BRA)

Summary of an Agda development

May 2026

Abstract

We present a fully formalized proof of both Gödel’s First and Second Incompleteness Theorems in a tree-based variant of Guard’s Basic Recursive Arithmetic (BRA), executed in Agda 2.8 with `--safe --without-K --exact-split`. The development contains **zero postulates and zero holes**; every per-file solo typecheck completes in under one second (with one historical exception, an $\sim 11\text{k}$ -line dispatcher cascade for *thmT*). The development is organized as a three-layer staged calculus (object terms / codes / object-level operations on codes; see Section 1.1), with explicit typed maps between layers and a code-quote convention $\ulcorner \cdot \urcorner$ for the Layer-2 trees that name Layer-1 syntactic objects.

The headline results are:

- *thm11* : *Deriv G* $\rightarrow$ *Deriv* $\perp$ (Gödel I: provability of the diagonal sentence implies $\perp$);
- *godelII* : *Deriv Con* $\rightarrow$ *Deriv* $\perp$ (Gödel II: provability of consistency implies $\perp$).

The development follows Guard’s 1963 lecture notes on recursive arithmetic exclusively. The intermediate construction is a single first-order computable function *constr5* : *Fun1* realizing Guard’s “*F(x)*” (page 17 of his notes); each step in the chain is given by an explicit equational derivation in BRA.

1 Introduction

The aim of the project is to give a feasible, ASCII-only, syntax-first Agda formalization of Gödel’s two incompleteness theorems, following Guard’s 1963 system of *Basic Recursive Arithmetic* [1]. The intent is to avoid historical baggage (Gödel numbering of Hilbert proofs, primitive recursive arithmetic with explicit numerals, “encoded” substitution as a Hilbert-system meta theorem) by working inside a small tree-based equational system from the outset.

1.1 Three-layer staging

The development is organized around three syntactically distinct layers, with explicit typed maps between them. Keeping these layers separate eliminates the level-conflations that plague Guard’s pen-and-paper notation (especially around “underlining” $\underline{x}$ for numerals; see Section 4.1).

Layer 1 — Object language (terms, formulas, derivations). The classical layer where Gödel I/II are eventually stated.

- *Term*: built from *O* (leaf), *var n*, *ap₁ f t*, *ap₂ g t₁ t₂*.

- Function symbols come in two arities: Fun_1 and Fun_2 , with primitive constants $\{I, Fst, Snd, Comp, Comp_2, Z\}$ and $\{Pair, Const, Lift, Post, Fan, IfLf, TreeEq, treeRec\}$.
- *Formula*: atomic equations $atomic (eqn \ell r)$, plus *not* and *imp*.
- *Deriv F*: an Agda inductive family of derivations. Constructors for the algebraic axioms (ax_*), the equality axioms (Guard’s Ax 4–7), three propositional axioms ($axK, axS, axNeg$, Lukasiewicz form), modus ponens (mp), substitution ($ruleInst$), and binary tree induction ($ruleIndBT$). Classical $axContrapos$ and $axExFalso$ are derived lemmas, not constructors.

Layer 2 — Code datatype. A separate use of *Tree* as the datatype of *codes* (“Gödel numbers,” but trees instead of integers).

- *Tree*: built from lf and $nd a b$.
- Typed encoding maps:

$$code : Term \rightarrow Tree, \quad codeFormula : Formula \rightarrow Tree,$$

$$codeF_1 : Fun_1 \rightarrow Tree, \quad codeF_2 : Fun_2 \rightarrow Tree,$$

each producing a tag-prefixed tree (e.g. $code (ap_2 g t_1 t_2) = nd \ tagAp_2 (nd (codeF_2 g) (nd (code t_1) (code t_2)))$).

- Reverse map (numeralisation):

$$reify : Tree \rightarrow Term$$

sending $lf \mapsto 0$ and $nd a b \mapsto ap_2 \ Pair (reify a) (reify b)$. Natural numbers are represented as right-spine trees: lf for 0, $nd lf n$ for the successor.

Layer 3 — Reflection (object-level operations on codes). Fun_1 and Fun_2 combinators that operate on codes *within* the object language, allowing BRA to reason about its own syntax internally:

- $cor : Fun_1$: the “code-of-numeral” map. Spec: $cor x = code (reify x)$ (BRA-derivable via $corOfReify$). See Section 4.2 for a naming proposal.
- $sub : Fun_2, subT : Fun_2$: coded substitution, lifting the meta-level $substF$ to operate on coded terms/formulas.
- $thmT : Fun_1$: the proof-checker. See Section 2.

Reflection theorem (encoding-soundness). The link between layers is provided by

$$encode : (P : Formula) \rightarrow Deriv P \rightarrow \Sigma Tree (\lambda y. Deriv (thmT (reify y) = reify (codeFormula P))).$$

Given any object-level derivation, *encode* produces an encoded proof tree y together with an object-level proof that $thmT$ on y evaluates to the code of P . Layer 3 reflects Layer 1 through Layer 2’s coding.

1.2 Notational convention: code-quotes

We use $\ulcorner E \urcorner$ throughout for “the code (Layer 2 tree) of E .” Concretely:

$$\ulcorner t \urcorner := \text{code } t \quad (t : \text{Term}), \quad \ulcorner F \urcorner := \text{codeFormula } F \quad (F : \text{Formula}).$$

Whenever a Guard-style expression mixes object-level and encoded material (e.g. $f(\underline{x})$), we keep the code-quote explicit: the formal object is the Tree $\ulcorner f(\underline{x}) = \underline{f(x)} \urcorner$, never the unquoted equation. See Section 4.1 for the full caveat.

2 The provability predicate $\text{thm}T$

The Gödel-style “ th ” provability predicate is realized in BRA by a single Fun_1 -functor:

$$\text{thm}T = \text{Rec stepProtoWrapped} : \text{Fun}_1.$$

Here stepProtoWrapped is a Fun_2 assembled from a 40-deep linear IfLf cascade (`BRA/Thm/StepProto.agda` and `BRA/Thm/ThmT.agda`, ~16k lines), one branch per primitive proof rule: ten equational axioms of Group I (`axI`, `axFst`, ..., `axKT`), eight Group II axioms (`axIfLfL`, `axIfLfN`, four `axTreeEq*`, `axGoodstein`), nine equality and propositional rules, and three rules of inference (`mp`, `ruleInst`, `ruleIndBT`).

The *key* property of $\text{thm}T$ is that on the encoded form of any proof tree it returns the encoded conclusion of that proof. This is the content of the *encoding-soundness* lemma $\text{encode} : (P : \text{Formula}) \rightarrow \text{Deriv } P \rightarrow \Sigma \text{ Tree } (\lambda y \rightarrow \text{Deriv } (\text{thm}T (\text{reify } y)) = \text{reify } (\text{codeFormula } P))$, proved by induction on the Deriv derivation in `BRA/Thm/ThmT.agda`.

2.1 Self-substitution closure

A second key property is that $\text{thm}T$ contains no free variable (it is a closed combinator):

$$\text{thClosed} : \forall x r, \quad \text{substF}_1 x r \text{thm}T = \text{thm}T$$

and that the encoding of $\text{thm}T$ is var-free:

$$\text{codeF1Th_noVar} : \forall x \text{repl}, \quad \text{codedSubst repl } (\text{code } (\text{var } x)) (\text{codeF}_1 \text{thm}T) = \text{codeF}_1 \text{thm}T.$$

Both are `refl` inside the abstract block where $\text{thm}T$ unfolds.

3 Gödel’s First Incompleteness Theorem (Theorem 11)

3.1 The diagonal sentence

Following Guard, define

$$\begin{aligned} F_{\text{pre}} &:= \neg(\text{thm}T(v_1) = \text{sub}(v_0)(v_0)), \\ j &:= \text{codeFormula } F_{\text{pre}} \quad (\text{a closed Tree}), \\ G &:= F_{\text{pre}}[v_0 := \text{reify } j] = \neg(\text{thm}T(v_1) = \text{sub } i \text{ } i), \end{aligned}$$

where $i := \text{reify } j$ and sub is the BRA-internal coded substitution function (`BRA/Sub.agda`). The sentence G has v_1 free; informally, G asserts of itself that no proof tree v_1 proves the formula whose code is $j[v_0 := i]$.

3.2 Theorem 11

Pulling apart the abstract closure (`BRA/Thm11Abstract.agda`), the precise statement is:

Theorem 1 (`thm11`, Gödel I). *thm11 : Deriv G → Deriv ⊥, where ⊥ := atomic (eqn trueT falseT) with trueT := O and falseT := ap₂ Pair O O.*

Sketch. From any $\pi : \text{Deriv } G$ extract its encoding (y_d, E_1) where $E_1 : \text{Deriv } (\text{thmT } (\text{reify } y_d) = \text{reify } (\text{codeFormula } G))$. A separate *diagonal lemma* `diagonal` (`BRA/Thm11Diagonal.agda`) yields $N : \text{Deriv } \neg(\text{thmT } (\text{reify } y_d) = \text{reify } (\text{codeFormula } G))$ from π , and $ax_{ExFalso}$ + two *mp*’s collapse E_1 and N to $\text{Deriv } \perp$. $\square$

The diagonal lemma is constructed inside an abstract block in `BRA/Thm11Diagonal.agda` using `thClosed`, `codeF1Th_noVar`, and `cc` (the local instance of the meta-equation `codedSubst (code i) (code v0) j = codeFormula G`, which is provable by induction on F_{pre} via `codeCommutesFormula` in `BRA/CodeCommutes.agda`).

4 Gödel’s Second Incompleteness Theorem (Theorem 14)

The second theorem is the more substantial construction. It says that the consistency formula *Con* entails $\perp$:

Theorem 2 (`godelIII`, Gödel II). *godelIII : Deriv Con → Deriv ⊥, where Con := ¬(thmT (v₀) = reify (codeFormula ⊥)) (consistency: “no proof, encoded as v₀, derives ⊥”).*

The construction follows Guard’s pages 16–17 step by step.

4.1 Caveat: only the *coded* form is a well-defined BRA object

A structurally crucial point, easy to miss in Guard’s notation, is that the expression

$$f(\underline{x}) = \underline{f(x)}$$

is **not** a BRA formula. Guard’s underline $\underline{(\cdot)}$ takes a value and returns its numeral term, an operation that is well-defined only at the meta-level for closed numerical inputs. For a variable x — which is precisely the case Theorems 12–14 actually need — neither $\underline{x}$ nor $\underline{f(x)}$ has any BRA referent: there is no operator in BRA’s term language that, given an arbitrary `Term`, returns “the numeral of its value.” No attempt at writing the equation directly as *atomic (eqn)* for a variable x succeeds, because the slots cannot be filled with BRA terms.

The only formal object that *does* correspond to Guard’s expression is its *code* — the Tree

$$\ulcorner f(\underline{x}) = \underline{f(x)} \urcorner,$$

which Definition 12 of `guard15.pdf` defines by stipulation. In our Agda development this Tree is built parametrically as `codeFTeq1 f x`, with *cor* x filling the slot for the code of x ’s numeral and *cor* $(f x)$ filling the slot for the code of $f(x)$ ’s numeral.

Consequence for Theorem 12’s statement. When we write

$$\text{Deriv } (\text{thmT } (D_f x) = \text{codeFTeq}_1 f x),$$

the right-hand side is a *specific Tree*, not a BRA formula. The colloquial reading “BRA-internally, $f(\underline{x}) = \underline{f(x)}$ ” is shorthand for “BRA proves that the proof-checker *thmT* applied to $D_f x$ equals the Tree named by Definition 12.” Throughout the rest of this section, every occurrence of $f(\underline{x}) = \underline{f(x)}$ (and the analogous $f(\underline{x}) = \underline{y}$ form used in Theorem 13) should be read as a name for the defined Tree, never as a free-standing BRA formula.

4.2 Naming remark: `cor` would be more honestly named `numeralCode`

The combinator $cor : Fun_1$ is named after Guard’s “cor” (Exercise 24 [7] of `guard15.pdf`), but its operational role in our staging is more specific. Recall that $cor\ x = code\ (reify\ x)$ provably in BRA. In the three-layer terminology of Section 1.1:

- $reify : Tree \rightarrow Term$ converts a Layer-2 tree to a Layer-1 numeral term.
- $code \circ reify : Tree \rightarrow Tree$ composes that with the Layer-1-to-Layer-2 encoding.
- cor is the Layer-3 *combinator* that realizes $code \circ reify$ inside BRA’s term language.

A cleaner name would therefore be *numeralCode* (or *numCode*): “the code of the numeral term naming x .” We retain cor in the Agda code for continuity with Guard, but the reader is encouraged to mentally substitute *numeralCode* throughout. In particular, $cor\ x$ is **not** the numeral of x ; it is one encoding-step further, namely the Layer-2 tree that codes the Layer-1 term that names x . This is the source of the level-mismatch worries in Sections 4.1–4.4.

4.3 Numerals: the asymmetry $f(\underline{x})$ vs. $\underline{f(x)}$

A crucial feature of the Theorem 12–14 chain is the *asymmetric* treatment of the two cor -occurrences in the encoded equation. In Guard’s notation [1] the underline $\underline{x}$ denotes the *numeral term* naming x — a Layer-1 closed term whose value is x (Section 1.1). Our Layer-3 combinator cor realizes the composite *code-of-numeral-term*, i.e. the Layer-2 tree corresponding to $\underline{x}$ via the Layer-1-to-Layer-2 encoding (see Section 4.2).

The two cor -slots in the conclusion of Theorem 12 play different roles:

- LHS slot $ap_1\ cor\ x$ in $\ulcorner f(\underline{x}) \urcorner$: encodes the input to f as a numeral-code, parametric in x . When x is a free Layer-1 variable, this slot is a literal syntactic $ap_1\ cor\ x$ Term.
- RHS slot $ap_1\ cor\ (ap_1\ f\ x)$ in $\ulcorner \underline{f(x)} \urcorner$: encodes the *value* $f\ x$ as a numeral-code. Here cor is composed with f , so the slot has the form “ $cor \circ f$ at x .”

The two trees disagree in general; they agree only after BRA proves them equal — which is exactly Theorem 12’s content.

Theorem 12 is precisely the internalisation of that equation: for every $f : Fun_1$ it builds a $D_f : Fun_1$ such that

$$Deriv\ (thmT\ (D_f\ x) = codeFTeq_1\ f\ x),$$

where the right-hand side $codeFTeq_1\ f\ x$ is the Tree defined by Guard’s Definition 12 as the code of $f(\underline{x}) = \underline{f(x)}$ — recall from Section 4.1 that this Tree, not the unquoted equation, is the formal object. Its outer shape is a *Pair* of the code-of-LHS (encoding “ $ap_1\ f$ applied to the code of x ’s numeral”, with $cor\ x$ in the inner slot) and the code-of-RHS (the code of $f(x)$ ’s numeral, which is $cor\ (f\ x)$).

In other words: BRA itself proves that the proof-checker $thmT$, applied to $D_f\ x$, equals the Tree named by Guard’s Definition 12. This is what licenses every subsequent step of Goedel II: it lets the internal $thmT$ predicate move between the syntactic and semantic sides of the encoded equation exactly where Guard’s pencil-and-paper argument tacitly does so.

A symmetric variant in which both sides are coded the same way ($code\ (f\ x)$ on both sides, say) collapses: the resulting trees do not align across the five steps of Theorem 14, and step 5 becomes underivable. The asymmetric form $f(\underline{x}) = \underline{f(x)}$ is therefore not a presentational choice but a load-bearing feature of the proof.

4.4 Reading the slot `cor u` in encoded shapes

A second subtlety hidden by Guard’s underline notation concerns the slot occupied by `cor u`. Pure `code` of a BRA term is defined by

```
code (ap1 f u) = nd tagAp1 (nd (codeF1 f) (code u)),
```

whereas Guard’s $\widehat{f(u)}$ uses `cor` in the inner slot:

```
code "f u_" = nd tagAp1 (nd (codeF1 f) (cor u)).
```

The notation tempts the reader to think this Tree is itself `code t` for some BRA term `t`, but in general it is not.

When the slot is filled by a syntactic `ap1 cor x` for a free variable `x`. This is the form that actually appears in the Theorem 14 chain (e.g. `encoded_th_x_at` in `BRA/Th14Step3.agda`, with `x : Term` held abstract, typically `x = var 0`). Read syntactically, the assembled expression IS `code t` for

$$t = ap_1 f (ap_1 cor x) :$$

the BRA term “ f applied to `cor x`” — `cor` kept syntactic, the variable position passed through unevaluated. This is the only sense in which $\widehat{f(u)}$ at a variable `u` literally encodes a single BRA term.

An example where the slot expression is not `code t` for any `t`. Trees of the same outer shape need not be codes of BRA terms. Take e.g.

```
nd tagAp1 (nd (codeF1 f) (nd tagAp1 lf)).
```

For `nd tagAp1 lf` to be `code t'`, we would need `t' = ap1 g s` with `nd (codeF1 g) (code s) = lf`, which is impossible (the payload is never `lf`). So the inner slot `nd tagAp1 lf` is not the code of any BRA term, and consequently the whole Tree above is not `code` of anything. Non-code Trees of this kind do arise as intermediate values inside the `thmT` dispatcher cascade — the encoding-shape is not protected against them by the type alone.

Bridging the two readings. At a closed numeric Tree `u`, `corOfReify` (`BRA/Cor.agda`) gives `cor u = code (reify u)` internally as a `Deriv`, so under that reading the assembled Tree equals `code (ap1 f (reify u))` — a *different* `t` (the term “ f applied to the numeral of `u`”) from the variable reading above. Theorem 12 is precisely the BRA-internal lemma that equates the two: the equation $ap_1 f (cor x) = cor (f x)$ is the internal version of “the syntactic-variable reading and the evaluated-numeral reading agree”.

4.5 Theorem 12 — the encoded “*th f*” clause

For every `Fun1`-functor `f`, Theorem 12 produces a `Fun1 Df` such that under the canonical encoding, `thmT` evaluated at `Df x` is provably equal to the encoded form of “ $f(num\ x) = num\ (f\ x)$ ”. Formally:

Theorem 3 (`thm12_F1`, parametric Theorem 12 for `Fun1`). $thm12_F1 : (f : Fun_1) \rightarrow Thm12_F1_Result\ f$ where the record `Thm12_F1_Result f` packages a `Df : Fun1` together with `proof : (x : Term) → Deriv (thmT (Df x) = codeFTeq1 f x)`. The conclusion `codeFTeq1 f x` is the literal Pair-shape encoded equation $ap_1 f (cor\ x) = cor\ (f\ x)$.

(Analogous statement for Fun_2 .) This is a structural recursion on f : there is one clause per Fun_1 / Fun_2 constructor (15 total), each in its own `BRA/Thm12/Parts/*.agda` file. The composite cases (*Comp*, *Comp*₂, *Lift*, *Post*, *Fan*, *treeRec*) dispatch to per-case modules; the recursive *TreeEq* case uses `ruleIndBT2` (2-tree-pair induction with diagonal cross-IHs) to establish closure. The whole theorem is in `BRA/Thm12.agda`, module `FromBridges`, and contains zero postulates / zero holes.

4.6 Theorem 13 — under hypothesis “ $f\ x = y$ ”

Building on Theorem 12, one produces under the hypothesis $f\ x = y$ the encoded form of “ $f(num\ x) = num\ y$ ”:

Lemma 4 (*thm13_F1*). *For every $f : Fun_1$ and every Theorem 12 result r_{12} for f , $thm13_F1\ f\ r_{12}\ x\ y : Deriv\ (f\ x = y) \rightarrow Deriv\ (thmT\ (D_f\ x) = codeFTeq_{1,Hyp}\ f\ x\ y)$ where the conclusion replaces the “ $cor\ (f\ x)$ ” slot of $codeFTeq_1$ by $cor\ y$. One line: `ruleTrans` on $r_{12}.proof\ x$ with $cong_R\ Pair$ on $cong_1\ cor$ of the hypothesis.*

4.7 The function `sb` beyond its specification

Guard ([1], p. 14, Exercise 24 [4]) specifies `sb` as a substitution function only on encoded arguments:

$$sb(J(\ulcorner T \urcorner, n), \ulcorner P \urcorner) = \ulcorner P[T/x_n] \urcorner.$$

That is, the lecture notes describe `sb` only when the first slot is the code $\ulcorner T \urcorner$ of a BRA term T . In our formalisation `sb : Fun2` is defined uniformly via the underlying tree-level substitutor `codedSubst` of `BRA/Term.agda`:

```

codedSubst : Tree -> Tree -> Tree -> Tree
codedSubst repl varCode lf          = lf
codedSubst repl varCode (nd a b) = boolCase (treeEq varCode (nd a b))
                                         repl
                                         (nd (codedSubst repl varCode a)
                                              (codedSubst repl varCode b))

```

The substituent `repl` here is just *any* `Tree`; nothing in `codedSubst` relies on `repl = code T` for some BRA term T . The corresponding spec (`BRA/Sb.agda`, `sb_def`)

```

ap2 sb (ap2 Pair (reify codeA) (reify (natCode n))) (reify codeP)
  = reify (codedSubst codeA (code (var n)) codeP)

```

holds for any `Tree codeA`, not just for `codeA = code T`. Guard’s lecture notes do not state this generalisation.

Crucial use in Theorems 13–14. In `BRA/Th14Step4.agda` the substituent slot is filled by `ap1 cor x` with $x : Term$ held abstract:

```

ap2 sb (ap2 Pair (ap1 cor x) varCode1T) (reify (codeFormula G)).

```

By Section 4.4 the value `cor x` at a closed numeric x is *not* `code T` for any BRA term T : it is a numeral encoding produced by `cor`’s recursion on the binary-tree shape. Yet `sb` processes it cleanly, walking `codeFormula G` and replacing each subtree equal to `code (var 1)` by the raw `Tree cor x`. The whole construction of *K-part*, *g-part*, and *constr5* in Theorem 14 (Section 2 below) depends essentially on this — the Definition 12 line 2 of *th* (`guard15.pdf` p. 15) substitutes a *numeral* into a coded formula, and a numeral is not the code of any BRA term.

4.8 Theorem 14 — the consistency-to-bot construction

This is the heart of Gödel II. The contract abstracted in `BRA/Thm14Abstract.agda` asks for:

1. a Fun_1 -functor constr5 such that $\text{thmT} (\text{constr5 } x)$ syntactically encodes $\perp$;
2. a parametric internal-implication

$$\text{step5} : (x : \text{Term}) \rightarrow \text{Deriv}((\text{thmT } x = \text{reify } (\text{codeFormula } G)) \text{ imp } (\text{thmT } (\text{constr5 } x) = \text{reify } (\text{codeFormula } G)))$$

The closure closureToG then applies $\text{ax}_{\text{Contrapos}}$ to step5 instantiated at v_1 , $\text{ruleInsts } \text{Con}$ at $\text{constr5 } v_1$, modus-ponenses to obtain $\neg(\text{thmT } v_1 = i)$, transports through $\text{subIleqcG} (\text{sub } i \ i = \text{cor } G)$ and G_{unfold} (propositional normal form of G), arriving at $\text{Deriv } G$. Composing with thm11 gives $\text{godelII} : \text{Deriv } \text{Con} \rightarrow \text{Deriv } \perp$.

4.8.1 Construction of constr5

Following Guard’s page 17, the construction relies on two pre-existing tautologies, encoded:

- $t := (v_0 = v_2) \text{ imp } ((v_1 = v_2) \text{ imp } (v_0 = v_1))$ (transitivity);
- $t' := (v_0 = v_1) \text{ imp } ((\neg(v_0 = v_1)) \text{ imp } \perp)$ (ex falso).

Both are derivable in BRA (`BRA/Thm14Numerals.agda`), and via encode they yield closed Trees $t\text{term}, t'\text{term}$ such that $\text{thmT} (\text{reify } t\text{term}) = \text{reify } (\text{codeFormula } t)$ (and analogously for t').

The chain is:

$$\begin{aligned} f_part(x) &:= \text{ruleInst } v_0 (th \ \hat{x}) (\text{ruleInst } v_1 (\text{sub } i \ i) (\text{ruleInst } v_2 \text{ cor } G \ t\text{term})), \\ g_part(x) &:= \text{mp}(\text{mp}(f_part(x), D_{\text{thmT}} x), D_{\text{sub}} i \ i), \\ K_part(x) &:= \text{ruleInst } v_1 (\text{cor } x) x, \\ h_part_skel(x) &:= \text{ruleInst } v_0 (th \ \hat{x}) (\text{ruleInst } v_1 (\text{sub } i \ i) \ t'\text{term}), \\ \text{constr5}_{\text{final}}(x) &:= \text{mp}(\text{mp}(h_part_skel(x), g_part(x)), K_part(x)). \end{aligned}$$

The substituents are the encoded mixed-form Pairs of Guard: $\widehat{th \ \hat{x}} := \text{ap}_2 \text{ Pair } (\text{reify } \text{tagAp}_1) (\text{ap}_2 \text{ Pair } (\text{reify } (\text{codeFormula } G)) (\text{reify } (\text{code } (\text{ap}_2 \text{ sub } i \ i))))$, and $\text{cor } G := \text{reify } (\text{code } (\text{reify } (\text{codeFormula } G)))$.

4.8.2 The five steps and their lemmas

Following Guard’s enumeration (`guard15.pdf` pages 16–17):

Step 1 — step1_l (`Th14Constr5.agda`). By Theorem 13 applied to thmT at $(x, \text{cor } G)$ under hypothesis $\text{thmT } x = \text{cor } G$, get $\text{Deriv } (\text{PrAtX } x \text{ imp } \text{thmT } (D_{\text{thmT}} x) = \text{codeFTeq}_{1, \text{Hyp}} \text{thmT } x \text{ cor } G)$.

Step 2 — step2_l (`Th14Step2.agda`). By Theorem 13 applied to sub at $(i, i, \text{cor } G)$ under closed hypothesis subIleqcG .

Step 3 — step3_l ($= g_part_l$) (`Th14Step3.agda`, `Th14Step3Canon.agda`). Three sb-applications on t via thmTSubLemma , canonicalized via $\text{subT_codeFormula_imp/atomic}$ and leaf preservation lemmas, then two mp dispatches at g_part_inner and g_part . Output: $\text{Deriv } (\text{PrAtX } x \text{ imp } \text{thmT } (g_part \ x) = th \ \widehat{\hat{x}} = \text{sub } i \ i)$.

Step 4 — K_part_l (Th14Step4Final.agda). One sb-application at v_1 on the diagonal sentence G_{norm} via $thmTSubLemma$ + the $PrAtX$ x hypothesis to swap $thmT\ x \mapsto cor\ G$, canonicalized via $subT_codeFormula_neg/atomic$ and leaf preservation. Output: $Deriv\ (PrAtX\ x\ imp\ thmT\ (\widehat{K_part\ x}) = th\ \hat{x} \neq sub\ i\ i)$.

Step 5 — $step5_l$ (Th14Step5.agda). Two outer mp dispatches at $constr5_inner$, final and $constr5_{\text{final}}$, with $step3_l$ feeding the inner antecedent and K_part_l standing in syntactically as the second- mp witness. The dispatcher’s syntactic projection of the consequent gives the encoded $\perp$. Output: $Deriv\ (PrAtX\ x\ imp\ thmT\ (constr5_{\text{final}}\ x) = reify\ (codeFormula\ \perp))$.

Auxiliary — $h_part_skel_l$ (Th14Step5HPart.agda). Two sb-applications on t' , mirror of step 3 (one fewer layer because t' has only two free variables). Output: $thmT\ (h_part_skel\ x) = ex\ falso$ form. Used as $step5_l$ ’s inner d_1 .

The headline composition (BRA/GoedelIIIFull.agda) is then:

```
r12_th  : Thm12_F1_Result thmT
r12_th  = FromBridges.thm12_F1 thmT

r12_sub : Thm12_F2_Result sub
r12_sub = FromBridges.thm12_F2 sub

module Step5 = BRA.Th14Step5.Th14Step5 r12_th r12_sub
module Final = BRA.GoedelIII.Compose Step5.constr5_final Step5.step5_l

godelIII : Deriv Con -> Deriv bot
godelIII = Final.godelIII
```

4.9 What Theorems 12–14 are doing, and where the notation hides things

The BRA chain involves two distinct encodings of trees as trees. Pulling them apart helps explain what each of Theorems 12–14 is actually doing, and why Guard’s compact notation can mislead.

Two encodings. Two functions are at play, and they are not the same:

- $cor : \text{Tree} \rightarrow \text{Tree}$, the *numeral encoding*. $cor\ x$ (also written $\underline{x}$) is the canonical numeral whose value is x , a tree of the form $S(S(\dots S(0)))$.
- $code : \text{Term} \rightarrow \text{Tree}$, the *syntactic encoding* of a BRA term, recording the term’s syntax tree.

The two operations agree on no input shape: $cor\ x$ collapses a tree to its numerical magnitude, while $code\ t$ preserves the structure of t as a syntax tree.

The notational gotcha: $code(f(\underline{x}))$ vs $code(\underline{f(x)})$. Two expressions that look symmetric in Guard’s compact notation are quite different syntactic objects:

- $f(\underline{x})$ is the *syntactic term* “function symbol f applied to the numeral $\underline{x}$ ”. Its encoding $code(f(\underline{x}))$ is unambiguously the code of a (closed) BRA term: f applied to the numeral of x .

- $\underline{f(x)}$ is the *numeral* whose value is $f(x)$ — a tree $S^n(0)$ for some n . Guard’s notation $\text{code}(\underline{f(x)})$ takes the encoding of *that numeral*, which is in general *not* the code of the term $f(x)$: the term $f(x)$ has shape “ f applied to x ”, while the numeral $\underline{f(x)}$ has shape $S^n(0)$. Completely different syntactic trees.

The compact $(\cdot)$ invites the reader to treat the two sides of an equation like $\widehat{f(\underline{x})} = \underline{f(x)}$ as “the same expression with the bar moved”. In fact the inner brackets quietly switch between two different encodings: a syntactic-term encoding on the left, a numeral encoding on the right. Theorem 12 is the BRA-internal proof that, despite the notational mismatch, these two trees evaluate to the same value at every f .

Theorem 12: a per-functor lift D_f together with naturality. For every Fun_1 functor f , Theorem 12 produces a primitive recursive $D_f : \text{Fun}_1$ together with an internal derivation

$$\text{thm}T(D_f x) = \widehat{f(\underline{x})} = \underline{f(x)}.$$

D_f is the “code-level cousin” of f : it takes numerals to numerals. The encoded equation says that D_f commutes with the numeral encoding, in the sense that running f on a numeral and taking the numeral of $f(x)$ produce the same value. Theorem 12 simultaneously *defines* the lift D_f and *proves* that the lift is faithful.

Theorem 13: lifting an external equation to an internal one. Given a value-level $\text{Deriv}(f x = y)$, Theorem 13 produces an internal derivation $\text{thm}T(D_f x) = \widehat{f(\underline{x})} = \underline{y}$. Read externally: “BRA proves $f(x) = y$ at the value level $\Rightarrow$ BRA also internally proves the corresponding encoded equation between numerals”. This is the Hilbert–Bernays necessitation rule D1 ($\vdash A \Rightarrow \vdash \text{Pr}(\ulcorner A \urcorner)$) restricted to equational A , applied on top of Theorem 12 to rewrite the $\underline{f(x)}$ side of the encoded equation as $\underline{y}$.

Theorem 14, step 5: internalised Gödel I at the diagonal G . step5_l derives, parametrically in x , the encoded internal implication

$$\text{thm}T x = \text{cor } G \rightarrow \text{thm}T(\text{constr5 } x) = \text{cor } \perp.$$

Read externally: “if x codes a proof of G , then $\text{constr5 } x$ codes a proof of $\perp$ ”, i.e. the encoded implication $\text{Pr}(\ulcorner G \urcorner) \rightarrow \text{Pr}(\ulcorner \perp \urcorner)$.

The textbook decomposition of this internal implication has three pieces, all combined by internal modus ponens:

1. One direction of the diagonal, $G \rightarrow \neg \text{Pr}(\ulcorner G \urcorner)$, BRA-derivable from the construction of G . Necessitating this gives an internal derivation of $\text{Pr}(\ulcorner G \urcorner) \rightarrow \text{Pr}(\ulcorner \neg \text{Pr}(\ulcorner G \urcorner) \urcorner)$.
2. Hilbert–Bernays clause D2 at $A := G$: $\text{Pr}(\ulcorner G \urcorner) \rightarrow \text{Pr}(\ulcorner \text{Pr}(\ulcorner G \urcorner) \urcorner)$. In BRA the realiser of D2 at G is just the proof-checker: given a tree x with $\text{thm}T x = \text{cor } G$, the verification “ x codes a proof of G ” is itself the evidence that $\text{Pr}(\ulcorner G \urcorner)$ holds.
3. Internal modus ponens combining the two: the conclusion $\text{Pr}(\ulcorner G \urcorner) \rightarrow \text{Pr}(\ulcorner \perp \urcorner)$.

constr5 is the explicit primitive recursive function that packages the three pieces into a single proof-tree transformer; the chain $f_part, g_part, K_part, h_part_skel$ in Section 4.3 above is its syntactic decomposition.

So the connection to D2 is real but modest: D2 is one of three ingredients, and its BRA realiser is just the proof-checker, not a deep theorem. Step 5 is *not* Löb’s theorem ($\Box(\Box A \rightarrow A) \rightarrow \Box A$); Löb is never used in the chain.

Modal-logic sidebar. For readers familiar with intuitionistic modal $S4$ in Davies–Pfenning’s style [3]: the picture above has a clean modal reading. Their typed calculus has two contexts (Γ for ordinary value variables and Δ for “code” or “boxed” variables) and a type $\Box A$ of codes for A , with $\mathbf{box} M : \Box A$ allowed only when M depends solely on Δ -variables. Under this reading: the *cor* operation is the unit $A \rightarrow \Box A$; Theorem 12 is the action of $\Box$ on a morphism (so D_f is $\Box f$); Theorem 13 is necessitation D1 for equations; Theorem 14 step 5 invokes D2 at G plus the diagonal. The notational gotcha above corresponds to the fact that $\mathbf{box} (f x) : \Box B$ is well-typed but $f (\mathbf{box} x)$ is a type error — the modal type system catches by inspection what BRA’s uniform Tree type lets the notation hide. The modal reading is a useful specification, not a shortcut: Davies–Pfenning’s $\Box$ is metatheoretic and gives no algorithmic content for D_f , *constr5*, or *thmT*. Gödel II proper requires the provability predicate to be a primitive recursive function definable inside BRA itself; the deep encoding is intrinsic to the theorem.

5 Methodology

5.1 Fast typecheck as a correctness oracle

Every per-file solo typecheck runs in well under one second (with the historical exception of BRA/Thm/ThmT.agda at ~ 19 s; this is the 16k-line dispatcher cascade and was already in place before the Gödel II push). The session followed a strict rule: *slow typecheck = mathematical mismatch, stop and re-architect*. In practice this prevented two specific traps:

1. Trying to feed Agda an opaque normalisation it cannot perform (e.g. unfolding *codeFormula G* at a parametric substituent position). The fix is always: add a single targeted refl-witnessable lemma inside the right **abstract** block (where the opaque definition unfolds), and use it as a black box outside.
2. Forcing structural recursion through opaque trees (*codedSubst/codedSubstT* on *codeF₁ thmT*, etc.). Same fix: prove the equation once inside the abstract block, expose only the type signature outside.

5.2 Architectural choice for *constr5*

The original *Th14Constr5Final.constr5* wraps *h_part* with two *additional* encoded-mp layers (*h_part_thxLayer*, *h_part*). Under the *thmT* dispatcher these would consume both antecedents of *h_part_skel*’s ex-falso shape, collapsing the value of *thmT (h_part x)* to $\perp$ before it can serve as antecedent to *constr5_inner*. The fix is to use *h_part_skel* directly in *constr5_final* (pure sb-chain, no internal mp’s), matching Guard’s intent on page 17 and the comment in *Th14Constr5.agda*. The two outer *mp*’s in *constr5_final* then consume the two antecedents exactly as Guard’s chain prescribes.

5.3 The Carneiro-lifted chain: simulating the deduction theorem in a Hilbert system

BRA’s *Deriv* is a Hilbert-style system: implication is a connective, but there is no native introduction rule that, given a derivation of ψ depending on a hypothesis φ , returns *Deriv* ($\varphi \text{ imp } \psi$). The

propositional axioms are Guard’s

$$axK = P \text{ imp } (Q \text{ imp } P), \quad axS = (P \text{ imp } (Q \text{ imp } R)) \text{ imp } ((P \text{ imp } Q) \text{ imp } (P \text{ imp } R)),$$

together with modus ponens. axS is Frege’s 1879 axiom.

In such a system the deduction theorem can be *simulated* (a technique attributed to Mario Carneiro in the metamath community): replace every theorem ψ_i in a derivation sequence by $\varphi \text{ imp } \psi_i$. If ψ_k follows from ψ_i and $\psi_j = \psi_i \text{ imp } \psi_k$ by modus ponens, then $\varphi \text{ imp } \psi_k$ follows from $\varphi \text{ imp } \psi_i$ and $\varphi \text{ imp } (\psi_i \text{ imp } \psi_k) = \varphi \text{ imp } \psi_j$ via axS and two applications of modus ponens. Frege’s axiom is precisely what licenses the lifting; the form was apparently understood in 1879 and lost in Russell’s and Hilbert’s later presentations.

The Theorem 14 chain uses this trick at the level of the running hypothesis

$$PrAtX\ x := (thmT\ x = cor\ G).$$

Each step lemma $step1_l, \dots, step5_l, K_part_l, h_part_skel_l$ is the $PrAtX\ x \text{ imp } (\dots)$ -lifted form of the underlying step equation; the suffix “ $_l$ ” (*lifted*) marks this throughout the codebase. Modus ponens between two adjacent step lemmas is mediated by axS . The reusable combinators sit in `BRA/Thm/EvalHelpers.agda` (header “Lift helpers (deduction-theorem combinators)”); the lifted Theorem 13 for singular functors is in `BRA/Th14Constr5.agda` (“Carneiro-lifted Theorem 14 chain”); the per-step proofs live in `BRA/Th14Step3.agda`, `Th14Step4Final.agda`, `Th14Step5.agda`, and `Th14Step5HPart.agda`. Theorem 12’s *Rec* / *RecP* cases use the same idea at *dispatcher granularity* — the per-tag dispatcher is what is lifted, not each individual equation (`BRA/Th12RecP.agda`, `BRA/TreeEqRef1Param.agda`, with helpers in `BRA/Thm/EvalHelpers.agda`).

Without this simulation the Theorem 14 development would not close: each step genuinely depends on $PrAtX\ x$, and a Hilbert-style *Deriv* offers no native means to discharge such a hypothesis. The Carneiro lifting is therefore not a stylistic choice but a structural component of the proof, on a par with the *cor/code* asymmetry of Section 4.3.

6 Statistics

6.1 File-by-file summary of the Gödel II push

File	LoC	Solo typecheck
<code>BRA/Th14SubTLeaf.agda</code>	374	0.18 s
<code>BRA/Th14Step3Canon.agda</code>	582	0.60 s
<code>BRA/Th14Step3.agda</code>	633	0.81 s
<code>BRA/Th14Step4Final.agda</code>	213	0.84 s
<code>BRA/Th14Step5HPart.agda</code>	394	0.84 s
<code>BRA/Th14Step5.agda</code>	305	0.76 s
<code>BRA/GoedelIIFull.agda</code>	50	0.71 s

6.2 Earlier infrastructure (already in place)

- `BRA/Thm/ThmT.agda`: 16k LoC, 40-deep dispatcher cascade, 19 s.
- `BRA/Thm12.agda`: structural Theorem 12 with one Parts file per constructor, 0.65 s.
- `BRA/Thm13.agda`: ~80 LoC, 0.1 s.

- BRA/Thm11.agda, Thm11Abstract, Thm11Diagonal: ~ 300 LoC total.
- BRA/Thm14Abstract.agda: ~ 250 LoC, the *closureToG* machinery.
- BRA/Thm14Constr5Final.agda, Th14StepHPre, Th14HUnfolds, Th14GPartUnfolds: ~ 700 LoC of Pair-of-Comp2 unfold lemmas.

6.3 Constraints maintained throughout

- `--safe --without-K --exact-split` on every file.
- ASCII only. No Unicode.
- No postulates. No holes. No `with`-abstraction. No dot patterns.
- No new *Deriv* axioms beyond Guard’s primitive list.

7 Discussion

The two Gödel theorems in BRA, side by side:

- **Gödel I.** If the diagonal sentence G is provable, then $\perp$ is provable. Equivalently (under the meta-assumption that BRA is consistent): G is unprovable.
- **Gödel II.** If the consistency formula Con is provable, then $\perp$ is provable. Equivalently: BRA cannot prove its own consistency.

The intermediate functions $constr5_{\text{final}}$ and the parametric step lemmas $step1_l\text{--}step5_l$, K_part_l , $h_part_skel_l$ are *first-order* in the sense of recursive-arithmetic definability: each is a closed combinator built from primitive Fun_1 / Fun_2 symbols, and each *Deriv* derivation is given by an explicit equational proof in the BRA system. No external soundness theorem is invoked.

7.1 Comparison with prior formalisations

To our knowledge the only previous machine formalisation of Gödel’s *Second* Incompleteness Theorem is Paulson’s Isabelle/HOL development of hereditarily finite set theory [2]. Other formalisations (O’Connor in Coq, Han in Lean, Carneiro’s metamath work) target Gödel’s *First* theorem only.

Paulson’s development uses HF set theory and full Gödel numbering of Hilbert-style proofs, with encoded substitution treated as an external soundness theorem. The present BRA development by contrast follows Guard [1]: the “provability predicate” $thmT$ is itself a primitive-recursive combinator built from $Rec + Fan + Lift + IfLf$; encoded substitution is performed by an explicit primitive recursive function $sub : Fun_2$; and the entire chain (Gödel I and II) stays first-order, no Hilbert-proof Gödel numbering, no external soundness theorem.

References

- [1] J. R. Guard, *Lecture notes on recursive arithmetic*, Applied Mathematics Division, Argonne National Laboratory, 1963.
- [2] L. C. Paulson, *A machine-assisted proof of Gödel’s incompleteness theorems for the theory of hereditarily finite sets*, Review of Symbolic Logic, vol. 7 no. 3, 2014, pp. 484–498.

- [3] R. Davies and F. Pfenning, *A modal analysis of staged computation*, Journal of the ACM, vol. 48 no. 3, May 2001, pp. 555–604.